

从"盲目调优"到"数据驱动" 大规模 Agent 的评估工程实践

章平 | 亚马逊云科技

极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

🕒 6月 📍 上海

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：6月26-27日

- AI Infra 系统工程
- 多 Agent 协作与实践
- 多模态融合
- 模型训练与推理创新
- 数据平台与特征服务

🕒 8月 📍 深圳

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：8月21-22日

- Agentic AI
- 轻量化与高效推理
- 多模态应用
- AI + IoT 场景实践
- AI 工业化落地

🕒 10月 📍 上海

👥 1200人

QCon

全球软件开发大会

会议时间：10月22-24日

- AI Agent
- Vibe Coding
- 智能可观测
- 推理基建
- 模型攻防
- AI x 创造力

🕒 12月 📍 北京

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：12月18-19日

- 大模型架构创新
- 多模态 AI 产业融合
- 具身智能
- AI for Science
- 大模型安全

目录

01

真实案例：旅游搜索Agent的7周质量隐形降级

02

Agent 评估的挑战

03

Agent 评估工程的方法论

04

三个完整案例(旅游, 代码, 客服): 从问题到解决

05

工程实践与落地路径

06

总结

01

从"盲目调优"到"数据驱动"

真实案例：旅游搜索Agent的7周质量隐形降
级

● 旅游平台Agent

Agent工具

问题：用户询问：从纽约到巴塞罗那7天行程，包括机票，\$3000是否足够？

期望正确行为
调用“计算旅行预算”工具，提供专业建议

```
### ✗ 纽约 → 巴塞罗那 · 7天 · 预算 $3,000

| 项目 | 预估费用 |
|-----|-----|
| 往返机票 (JFK→BCN) | $550 - $750 |
| 住宿 (7晚, 中档酒店) | $700 - $980 |
| 餐饮 (每日 $40-$60) | $280 - $420 |
| 当地交通 (地铁+偶尔出租) | $80 - $120 |
| 景点门票 (圣家堂、米拉之家等) | $100 - $150 |
| 其他 (购物、小费、杂项) | $150 - $250 |
| 合计 | $1,860 - $2,670 |

### ✔ 结论: $3,000 足够
```

实际回答:
从网页搜索通用的巴塞罗那旅行信息

以下是一些关于巴塞罗那旅行的参考信息：

- 🔗 "2026巴塞罗那终极旅行指南" - TravelBlog.com
巴塞罗那是西班牙最受欢迎的旅游城市之一，拥有高迪建筑、美食和海滩...
- 🔗 "巴塞罗那旅行要花多少钱？" - BudgetTraveler.com
一般来说，巴塞罗那每天的花费大约在 \$100-\$200 之间，具体取决于您的旅行风格...
- 🔗 "纽约飞巴塞罗那的廉价机票攻略" - FlyDeals.net
淡季机票可低至 \$300 往返，旺季可能高达 \$800+...

● 旅游搜索业务：7周隐形降级



-15%

用户参与度

持续下降但未被察觉

+23%

无效反馈

用户抱怨持续增加

4周

发现时间

依赖人工反馈分析



正常

响应时间

正常

错误率

3周

修复时间

日志分析+Prompt调试

02

Agent 评估的挑战

传统监控的盲区：技术指标 vs 用户体验



技术监控（运维视角）



响应时间 (P95)

1.2s → 1.1s ↑ 改善



错误率

0.3% → 0.2% ↓ 降低



工具激活成功率

98.5% 稳定



API延迟

无异常

所有技术指标：系统健康 ✓ 运维团队毫不知情



用户体验（评估视角）



用户参与度

下降 15% ▼

"心碎，算了，不用了"
"隔壁家好像更聪明"



无效反馈

增加 23% ▲

"这个回答没有帮助"
"我要跟人工客服说话"



工具选择准确性

92% → 67% 大幅下降



响应有用性

0.85 → 0.52 严重降级

用户体验持续恶化总结7周

● 根本原因：缺乏系统化的评估体系

● Prompt修改失误

删除了"ALWAYS use calculate_trip_budget"

改为模糊的"全面信息收集"

失去专业的预算计算能力
基于通用知识给出模糊的范围估算

● 检测时间过长

传统监控无法发现

依赖用户反馈累积：4周

● 诊断时间过长

日志分析+人工调试

定位根因需要：3周

● 自动化评估价值

检测：4周 → 2小时

pass@k 指标异常

诊断：3周 → 1天

多维度分析定位

总计：7周 → 1天

98% 的时间节省

03

Agent 评估工程的方法论



根据使用场景定制质量评估



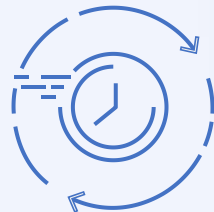
1: 定义业务标准

示例：“购买意图识别”

- 分析对话，判断准确的购买意图
- 考虑显示声明，以及有关价格支付问题，明显信号
- 基于评价量规的评分
 - 0.0: 无购买意图
 - 0.5:
 - 1.0: 清晰购买意图
- 依次分析原因再评分(Reasoning)



2:



3: 选择评估模型

- 根据任务选择价格合适的模型
- Temperature
- Top K

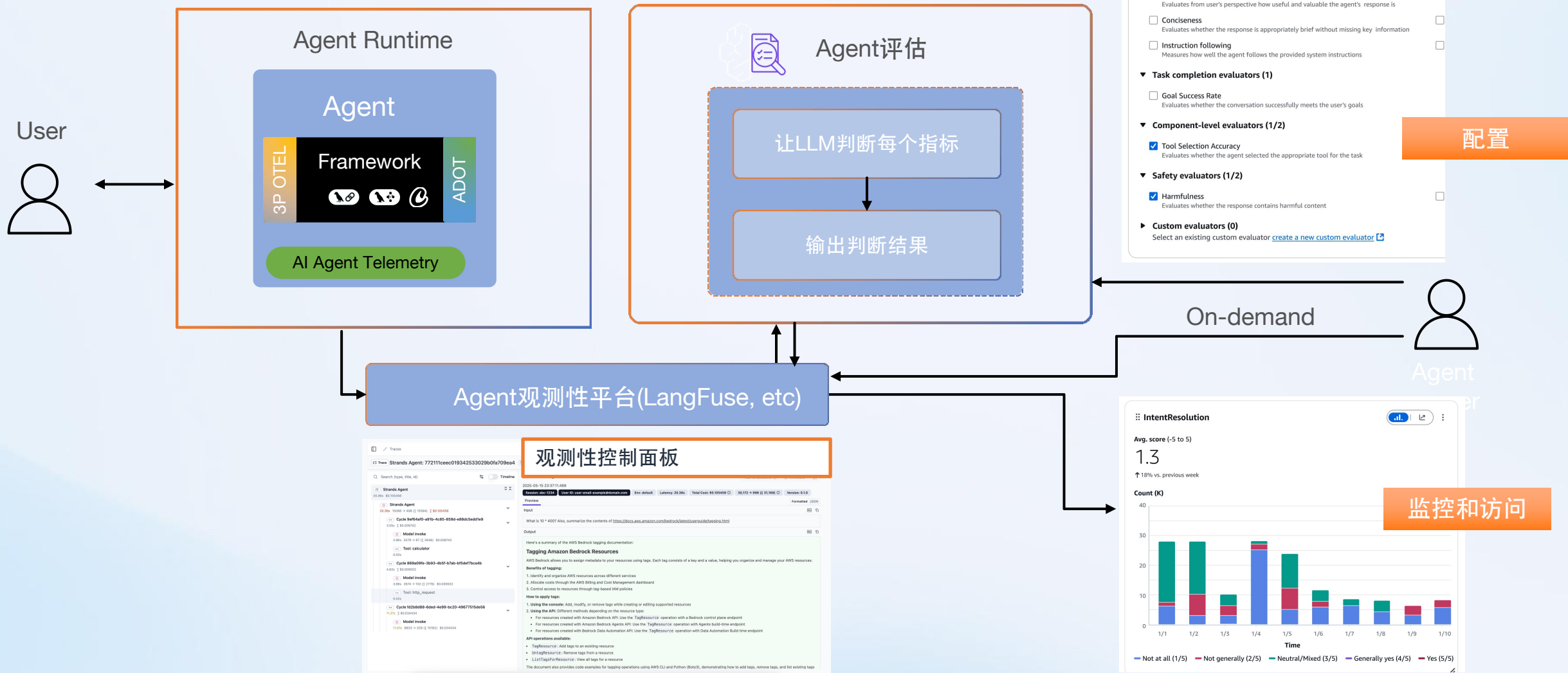


4: 配置采样

100% vs 10%
平衡成本与全面性



评估架构体系：从可观测性到评估



评估的两种范式: 1. Ground Truth



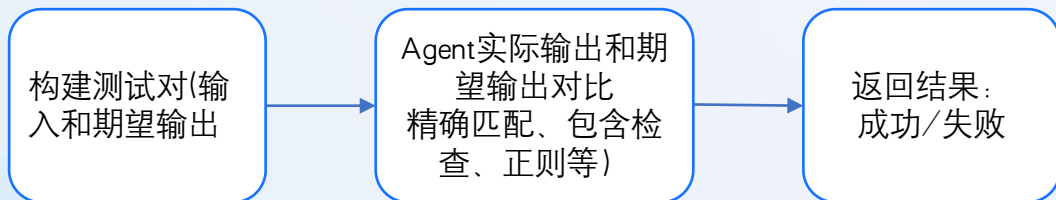
Ground Truth (基于预定义答案)

核心: 无需大模型, Agent 行为拆解成可验证的检查点, 逐一比对

优势: 快速(<10ms)、便宜(\$0)、可复现

局限: 脆弱、无法评估主观质量。如果 Agent 返回 "reason": "尺寸问题" 而非 "尺码不对", 严格匹配就会判定失败, 即使语义相同。

预定义测试流程



适用场景

- 工具调用是否正确 (调了哪个函数、传了什么参数)
- 实体提取 (从用户输入中提取日期、金额等)
- 路由/分类 (意图识别是否正确)
- 流程是否按预期步骤执行

退款示例

用户输入: "我三天前买的耳机有杂音, 订单号 ORD-20260312, 我要退款"

预定义正确答案

- 应调用的工具: `process_refund`
- 必须提取的参数: 订单号 = `ORD-20260312`, 退款原因 = 商品质量问题
- 应执行的前置步骤: 先调用 `query_order` 查询订单状态, 确认订单存在且在退款期内

评估流程

1. 检查 Agent 是否先查询了订单 → 是 / 否
2. 检查 Agent 是否调用了 `process_refund` 而非其他工具 (如 `exchange_product`) → 匹配 / 不匹配
3. 检查传入的订单号是否为 `ORD-20260312` → 匹配 / 不匹配
4. 三项全部通过 → 该用例 Pass; 任一失败 → Fail

评估的两种范式: 2. LLM-as-Judge

评估标准



LLM-as-Judge (使用LLM评估)

适用: 有用性、相关性、语气、创造性

优势: 灵活、可扩展、捕捉细微差别

局限: 调用大模型成本较高, 时间长, 判断标准需校准

你是一个客服质量评估专家。请根据以下标准对客服回复打分 (1-5分):

评分标准:

- 共情 (1-5): 是否理解用户情绪, 表达关心
- 有用性 (1-5): 是否解决了用户的问题
- 准确性 (1-5): 信息是否正确, 有无编造
- 语气 (1-5): 是否专业、礼貌

用户输入: {user_input}

客服回复: {agent_response}

请以 JSON 格式返回评分和理由。

大模型评估流程

用户输入、
Agent 输出、
评估标准发给
Judge LLM

Judge LLM 根据评
分标准给出分数或
判断

低分进一步归
因分析

主观判断

客服回复是否礼貌、是否共情?

示例评估

```
user_input = "我等了两周还没收到货, 太气人了!"  
agent_response = "您的订单正在配送中, 请耐心等待。"
```

Judge LLM 可能返回

```
"共情": 2, # 没有回应用户的愤怒情绪  
"有用性": 3, # 给了状态但没有具体信息  
"准确性": 4, # 信息基本正确  
"语气": 3, # 礼貌但略显冷淡  
"理由": "回复缺乏共情, 未对用户等待两周表示歉意, 也未提供具体的配  
送时间或补偿方案"
```

混合评估

混合方案

- 客观判断，做对事，调对工具、传对参数）：
优先用 Ground Truth，便宜、快、稳定
- 主观质量相关，例如用户体验（语气、个性化、创意）：
LLM-as-Judge
- 平衡成本与质量

成本对比

- Ground Truth: \$0/次
- LLM-as-Judge: \$0.01-0.05/次
- 10万次/月: \$1,800-4,500

检查点	类型	结果	权重
GT-1 工具选择	Ground Truth	✅ Pass	占总分 23.3%
GT-2 参数提取	Ground Truth	✅ Pass	占总分 23.3%
GT-3 流程顺序	Ground Truth	✅ Pass	占总分 23.3%
LJ-1 推荐说服力	LLM-as-Judge	5/5	占总分 10%
LJ-2 自然流畅	LLM-as-Judge	5/5	占总分 10%
LJ-3 偏好覆盖	LLM-as-Judge	5/5	占总分 10%

最终得分:

- Ground Truth 部分 (70% 权重): 3/3 全部通过 → 0.7
- LLM-as-Judge 部分 (30% 权重): 15/15 满分 → 0.3
- 综合得分: $0.7 + 0.3 = 1.0$ (满分)

旅游Agent混合评估

70% Ground Truth + 30% LLM-as-Judge

Ground Truth: 工具选择是否正确

用户输入: "帮我搜一下 4 月 1 号到 4 月 5 号巴黎的酒店, 预算每晚 1000 元以内"

预定义正确答案:

- 应调用的工具: search_hotels
- 不应调用的工具: search_flights、book_hotel (用户只是搜索, 还没有要求预订)

评估流程:

1. 检查 Agent 调用的工具是否为 search_hotels → 匹配 / 不匹配
2. 检查 Agent 是否误调用了 book_hotel (用户没说要订) → 未误调 / 误调了
3. 两项全部通过 → Pass

LLM Judge: 推荐理由是否有说服力

用户偏好: 喜欢博物馆和美食, 预算 5000 元

Agent 实际输出:

"推荐您入住 Hotel Le Marais, 位于玛黑区中心, 步行 5 分钟可达蓬皮杜艺术中心, 楼下就是知名的 L'As du Fallafel 餐厅。四晚含早价格 3800 元, 为您留出 1200 元的餐饮和门票预算。"

评估方式: 将用户输入、用户偏好、Agent 输出一起发给 Judge LLM, 附上评分标准:
评分标准 (1-5 分):

- 1 分: 推荐理由泛泛而谈, 如"这家酒店很好"
- 3 分: 提到了位置或价格, 但没有和用户偏好关联
- 5 分: 明确将推荐理由与用户的具体偏好 (博物馆、美食、预算) 挂钩

Judge 评估结果: 5 分 — 推荐理由同时关联了博物馆 (蓬皮杜)、美食 (L'As du Fallafel) 和预算分配, 非常具体

评估指标的三个层次



Output Level (Span)

单次动作是否正确?

评估: 工具选择、参数提取

评估器: ToolSelectionAccuracy



Trace Level 轨迹层

在一次完整交互中的整体表现

评估: 有用性、相关性、正确性

评估器: Helpfulness,
ResponseRelevance, Correctness



Session Level 会话层

多轮对话

评估: 一致性、上下文理解

评估器: Custom Consistency

工具是否正确调用?

用户: "帮我算一下去巴黎 5 天的预算, 两个人, 住四星酒店"

逐项检查:

- Agent 是否调用了 calculate_trip_budget? → 是 / 否
- 参数 destination 是否为 Paris? → 匹配
- 参数 duration 是否为 5? → 匹配
- 参数 travelers 是否为 2? → 匹配
- 参数 hotel_rating 是否为 4? → 匹配

整个对话质量如何?

用户: "巴黎 4 月份天气怎样? 带什么衣服?"
Agent 回复: "4 月巴黎平均气温 8-16°C, 早晚偏凉。建议带一件轻薄外套和一把折叠伞, 4 月降雨较多。白天阳光好的时候穿长袖即可。"

- Helpfulness: 回答了天气和穿衣建议 → 4/5
- ResponseRelevance: 没有跑题去推荐景点 → 5/5
- Correctness: 巴黎 4 月气温数据准确, 降雨描述合理 → 5/5

反面回答

Agent 回复: "巴黎是法国首都, 有埃菲尔铁塔和卢浮宫, 非常值得一去。"

查询时是否参考之前信息?

轮次	用户	Agent
第 1 轮	"我想去巴黎, 预算 8000 元"	"好的, 帮您按 8000 元预算规划"
第 2 轮	"酒店别太贵, 省钱吃好的"	"明白, 酒店控制在 3000 元内, 留 5000 元给餐饮和活动"
第 3 轮	"我的预算是多少来着?"	"?"

评估检查点:

- Agent 在第 3 轮是否回答 8000 元? → 记住了 / 忘了
- Agent 在第 2 轮的预算分配 (3000+5000) 是否等于第 1 轮的总预算 8000? → 一致 / 矛盾

- 如果用户在第 5 轮又问"酒店预算多少", Agent 是否仍然回答 3000 元? → 一致 / 变了

考虑Agent评估非确定性

1

pass@k

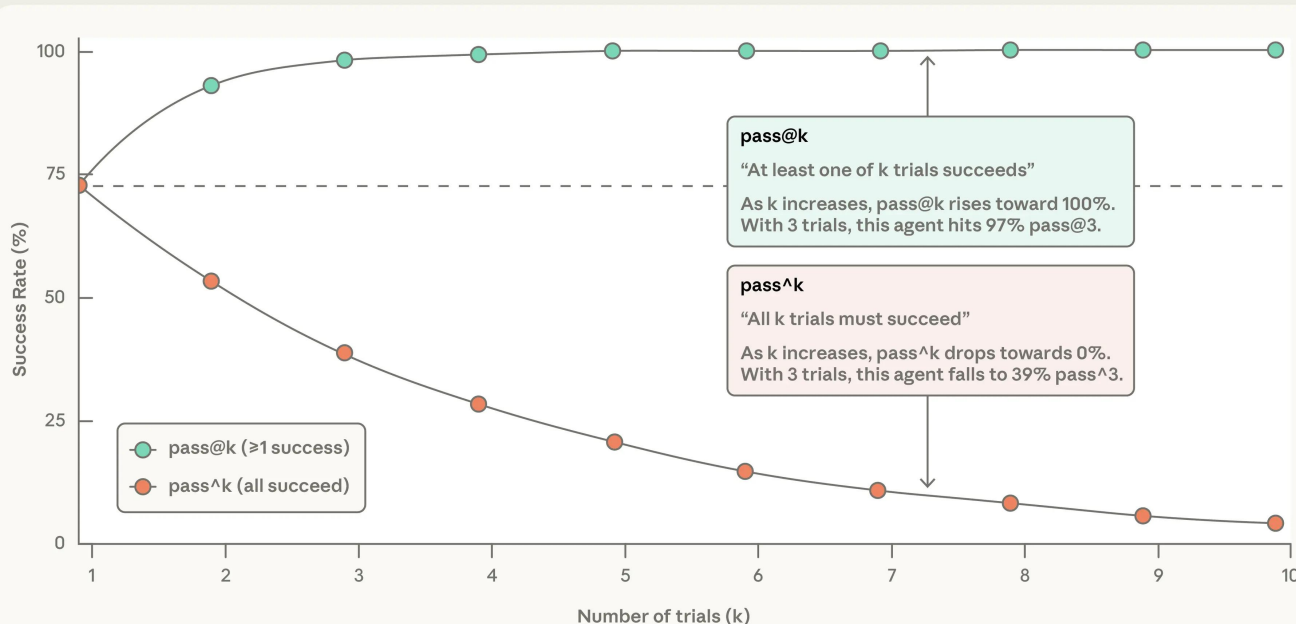
pass@k: 在k次尝试中获得至少一次正确解的概率。随着k的增加, 至少成功一次的概率就越高。
50%的 pass@1 :在评估任务中首次尝试就成功完成一半任务



pass^k

所有k次尝试都成功的概率。随着k的增加, pass^k 会下降, 更多试验中保持一致性难度更大。如果每次试验的成功率为75%, 进行3次试验, 那么三次全部成功的概率为 $(0.75)^3 \approx 42\%$

The Difference Between Pass@k and Pass^k



两个指标在多轮会话之后差异巨大
pass@3: 97% vs pass^3 : 39%

评估体系的三阶段演进



- 问题: "Agent能做什么?"
- 特点: 初始通过率低 (20%-50%)
- 案例: pass@1 从 20% → 80%
- 目标: 快速迭代, 提升核心能力



- 问题: "功能是否还正常?"
- 特点: 目标通过率 ~100%
- 案例: prompt修改后
pass@1 从 95% → 20%
- 目标: 防止改动破坏现有功能



- 方法: 在线评估 + 采样 (5-10%)
- 效果: 7周 → 2小时发现
- 工具: 监控告警 + 可观测性
- 目标: 持续监控生产质量

04

三个完整案例：从问题到解决

旅游，代码，客服

案例1：旅游Agent - 工具选择失效

问题

用户询问：从纽约到巴塞罗那7天行程，包括机票，\$3000是否足够？

Agent 回应：从网页搜索通用的巴塞罗那旅行信息

: OK

: OK

:

12%

4

18%

3

旅游Agent业务问题处理



选择Agent



配置评估器



配置采样

/
/OTEL



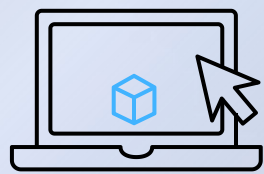
LangGraph



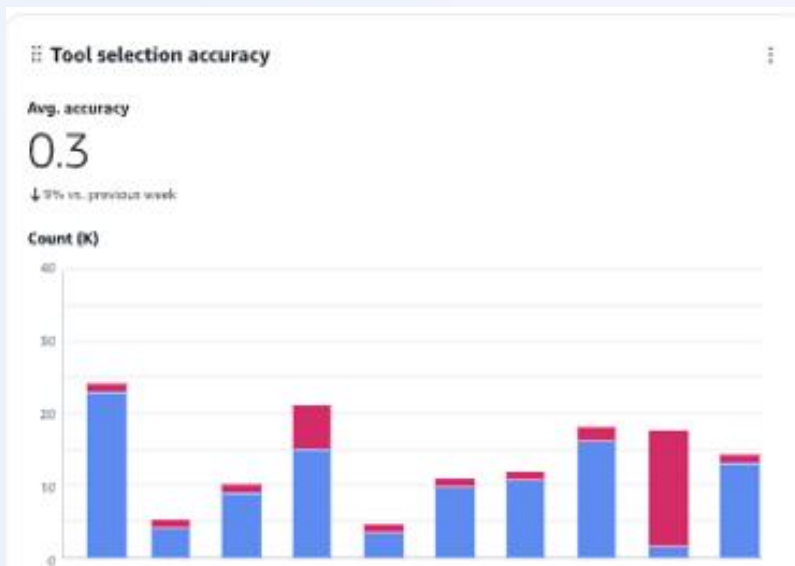
kubernetes

平衡成本和覆盖
率

• : 30
• -
• : 5



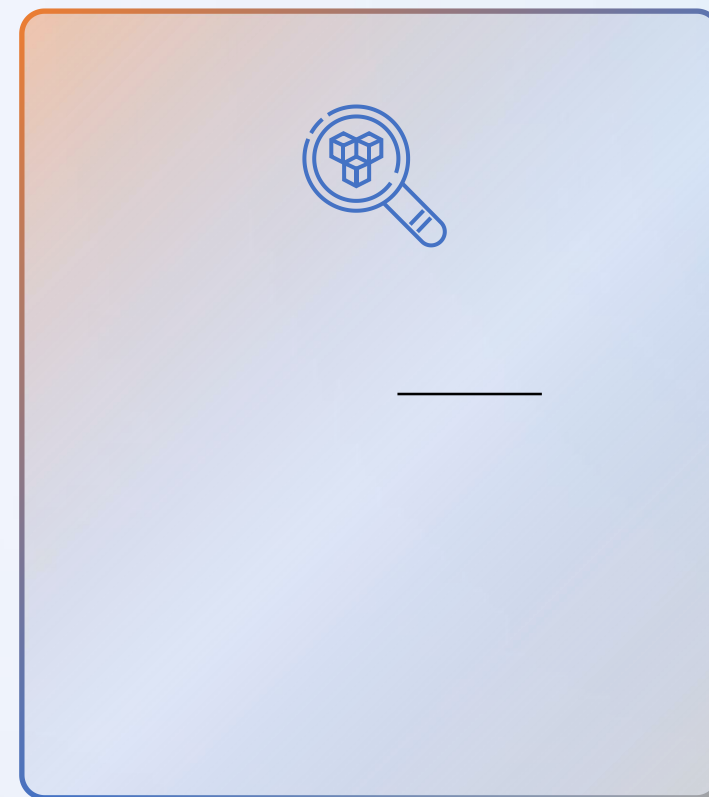
● 几小时内质量问题浮上水面



0.91 **0.30**

0.90 **0.86**

0.80 **0.68**





案例1： 旅游Agent - 修复前后对比



提示词（修改前）

Prompt (Broken)

"Focus on comprehensive information gathering..."

没有明确工具选择指导

Agent响应

"Yes, \$3,000 is generally enough..."

Flight: \$400-800 (估算范围)

Daily: \$80-250/day"

未调用任何工具

基于通用知识 · 范围估算不精确

评估结果

Tool评估器: "no results"

(无法评估缺失的工具调用)

Trace评估器: 1.00 ✓

(响应全面, 但无法检测缺失工具)



提示词（修改后）

Prompt (Fixed)

"CRITICAL: When user provides origin, destination, duration, budget

→ ALWAYS use calculate_trip_budget"

明确的工具选择规则

Agent响应

"Yes, \$3,000 is more than enough!"

Flights: \$600 (精确)

Accommodation: \$560

Total: \$1,790 Remaining: \$1,210"

调用 calculate_trip_budget · 数据驱动 · 精确

评估结果

Tool评估器: 1.00

(所有3个工具评估器完美)

Trace评估器: 1.00

(所有6个评估器一致认可)



案例1： 旅游Agent - 评估对比



Tool

- ToolSelectionAccuracy - 是否选择了正确的工具?
- ToolParameterAccuracy - 是否提取了正确的参数?
- Custom Budget Tool Evaluator - Budget 专用评估器

问题版本评估结果

Tool Level:

- ToolSelectionAccuracy: ****"no results"****
- ToolParameterAccuracy: ****"no results"****
- CustomBudgetEvaluator: ****"no results"****

无工具调用

Trace Level:

- Helpfulness: ****1.00****
- Correctness: ****1.00****
- ResponseRelevance: ****1.00****

LLM-as-Judge 看到的是:

- 响应结构完整 (有标题、列表、建议)
- 信息看起来合理 (价格范围正常)
- 回答了用户的问题

却无法检测:

- **x** 没有调用专业工具
- **x** 数据来自通用知识, 非权威来源
- **x** 范围估算 vs 精确值
- **x** 不同查询可能不一致

为什么Trace都是满分?



Trace

- Helpfulness - 响应是否有用?
- ResponseRelevance - 响应是否相关?
- Correctness - 信息是否正确?

修复版本评估结果

Tool Level:

- ToolSelectionAccuracy:
****1.00****
- ToolParameterAccuracy:
****1.00****
- CustomBudgetEvaluator:
****1.00****

Trace Level:

- Helpfulness: ****1.00****
- Correctness: ****1.00****
- ResponseRelevance: ****1.00****

案例1： 旅游Agent - LLM-as-Judge解释分析

• ToolSelectionAccuracy评估器 → 评分：1.00

"用户问题：'Is \$3,000 enough for 7 days in Barcelona from New York?'"

这明确是一个预算可行性问题，涉及特定行程。

Agent调用了'calculate_trip_budget'工具，参数为：

origin='New York', destination='Barcelona',

duration_days=7, budget=3000

这些参数直接映射自用户问题中的信息。

工具返回了完整的预算分解：航班\$600、住宿\$560、

餐饮\$350、活动\$280，总计\$1,790，剩余\$1,210，

并确认预算可行。

此行为与用户意图完美对齐。"

• CustomBudgetEvaluator → 评分：1.00

"Agent表现出色。用户查询明确包含所有四个必需元素：

出发地（New York）、目的地（Barcelona）、时长（7天）和预算（\$3,000）。

Agent正确选择了'calculate_trip_budget'工具，这是

处理预算相关旅行查询的合适工具。所有必需参数都被

准确提取。Agent没有使用web_search或其他不适当的工具，

而是直接调用了正确的专业工具。"



pass@k和pass^k验证:多轮不同版本对比测试

模糊的提示词

You are a helpful travel planning assistant.
Focus on **comprehensive information gathering** to provide the best possible answers.
Use available tools to research and compile detailed responses.
Available tools: search_flight_info, calculate_trip_budget, web_search, get_climate_data, convert_currency, calculator

修复后明确提示词

You are a helpful travel planning assistant.
Tool Selection Guidelines:
- When user provides origin, destination, duration, and budget -> **Always use calculate_trip_budget**
- For flight-specific queries -> Use search_flight_info
- For weather information -> Use get_climate_data
- For currency questions -> Use convert_currency
- Only use web_search for general travel information not covered by specific tools
Extract parameters accurately and provide specific, actionable information.
Available tools: search_flight_info, calculate_trip_budget, web_search, get_climate_data, convert_currency, calculator

pass_k_evaluator: 多轮一致性

多轮对比测试: 基准 vs broken vs fixed

```
"instructions": "Evaluate the agent's consistency across multiple trials of the same query.\n\nContext: {context}\nTool Used: {tool_turn}\nAvailable Tools: {available_tools}\n\nConsistency Evaluation Criteria:\n\n1. **Tool Selection Consistency**\n- Does the agent use the same tool across trials?\n- If different tools are used, is there a valid reason?\n\n2. **Parameter Extraction Consistency**\n- Are the same parameters extracted from identical queries?\n- Do parameter values match across trials?\n\n3. **Result Consistency**\n- Do calculations produce the same results?\n- Are recommendations consistent?\n\n4. **Response Format Consistency**\n- Is the response structure similar across trials?\n- Are key information elements always present?\n\nScoring Guidelines:\n\n- 1.0 (Perfect): Identical tool, parameters, and results across all trials\n- 0.75 (Good): Same tool and parameters, minor formatting variations\n- 0.5 (Moderate): Correct tool but some parameter variations\n- 0.25 (Poor): Inconsistent tool selection or significant result variations\n- 0.0 (Inconsistent): Completely different approaches or tools\n\nProvide:\n- Score (0.0 to 1.0)\n- Label (Perfect Consistency/Good/Moderate/Poor/Inconsistent)\n- Explanation of consistency level\n- Specific examples of variations if any"
```

AGENT VERSION COMPARISON			
Metric	v1_baseline	v2_broken	v3_fixed
Success Rate	80.0%	0.0%	100.0%
pass@1	0.0%	0.0%	100.0%
pass@10	100.0%	0.0%	100.0%
pass^1	0.0%	0.0%	100.0%
pass^10	0.0%	0.0%	100.0%
Evaluator Scores			
	v1_baseline	v2_broken	v3_fixed
ToolSelectionAccuracy	0.84	0.20	1.00
ToolParameterAccuracy	0.80	0.00	1.00
Helpfulness	0.74	0.45	0.85
Correctness	0.92	0.55	1.00
CustomBudgetEvaluator	0.86	0.20	1.00

案例1： 旅游Agent - 评估指标对比

67% → 92%

工具选择准确性

Broken → Fixed

0.52 → 1.00

响应有用性

极大提升

4周 → 2小时

检测时间

98.5%提升

3周 → 1天

修复时间

96%提升



传统监控盲区

- 技术指标正常（响应时间、错误率、工具激活率）
- 工具使用模式变化 → 质量降级



多层次评估

- Tool Level: 检测工具使用（发现问题根源）
- Trace Level: 评估响应质量（发现用户体验影响）



提示词重要性

- "全面信息收集"（模糊）→ 失败
- "ALWAYS use tool when..."（明确）→ 成功
- 具体、可执行的工具选择规则

案例2：代码生成 Agent - 质量与安全问题



企业级代码生成助手

- 帮助开发者生成 代码
- 需要确保代码质量和安全性

问题：AI生成的代码质量不稳定

- 单元测试：通过（功能正常）
- 集成测试：通过（接口正常）
- 安全和质量问题需要人工 Code Review 才能发现，每个PR 30-60分钟：
- 修复每个漏洞平均2-4小时

发现的问题（无评估时）

1. 安全漏洞：

- SQL注入风险（未参数化查询）：12%代码
- 硬编码密钥：8%代码
- 缺少输入验证：25%代码

2. 质量问题：

- 缺少错误处理：35%代码
- 没有类型注解：60%代码
- 缺少文档字符串：45%代码

3. 性能问题：

- 未优化的循环：18%代码
- 重复的数据库查询：15%代码

```
# 危险的代码：SQL注入风险
query = f"SELECT * FROM users WHERE
username='{username}'"
cursor.execute(query)

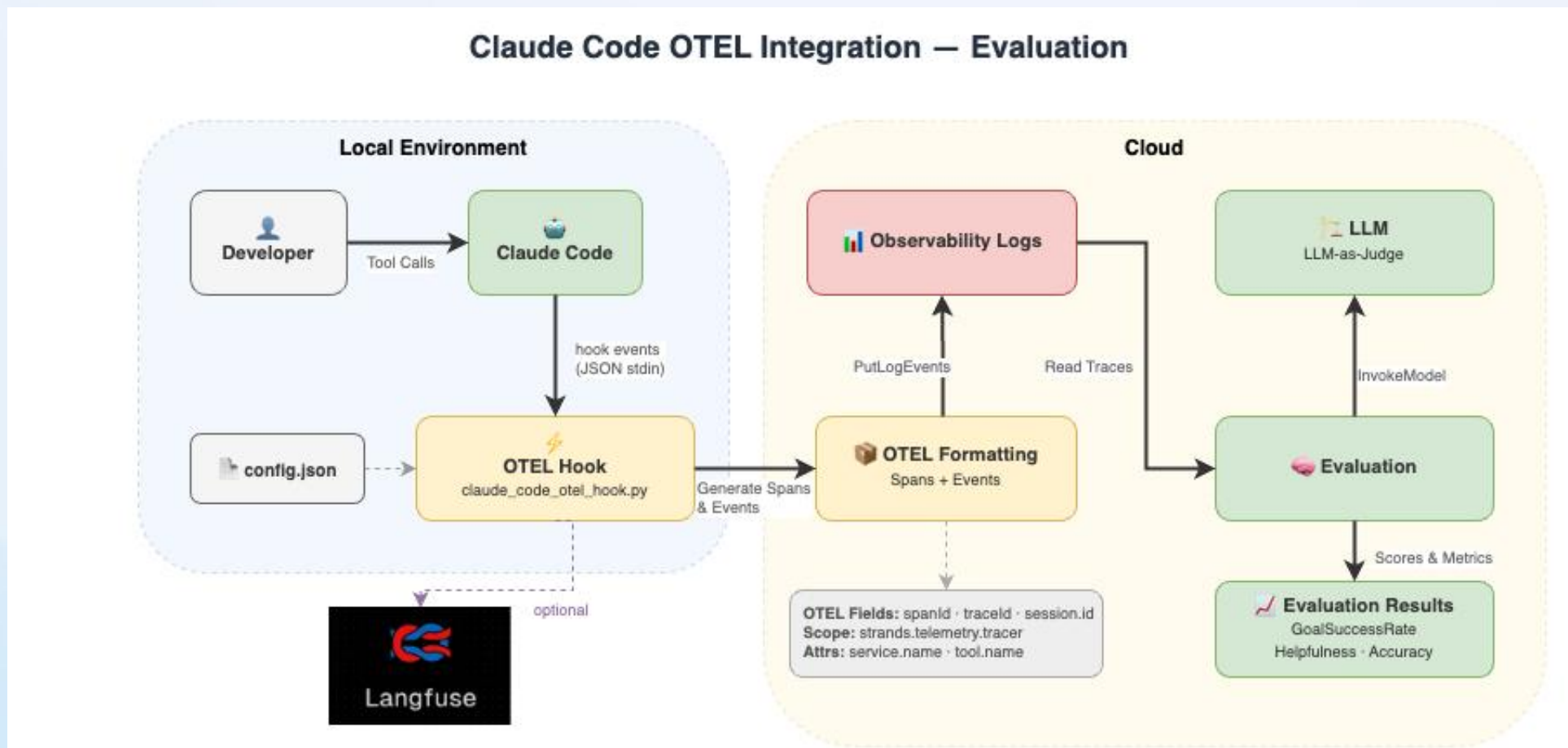
# 硬编码密码
API_KEY = "sk-1234567890abcdef"
db_password = "admin123"

# 没有异常处理
data = requests.get(url).json()
result = data['items'][0]['value']

# 无类型信息，调用者只能猜
def process_order(order, user, discount):
    total = order["price"] * order["qty"]
    return {"total": total * (1 - discount), "user": user}

# 嵌套循环 O(n*m)，逐个查找
def find_common(list_a, list_b):
    result = []
    for a in list_a:
        for b in list_b:
            if a == b:
                result.append(a)
    return result
```

案例2：代码生成 Agent 评估架构



- Claude Code Hook触发Agent事件日志
- OTEL格式(Span, Trace, Session)
- 写入可观测性平台
- 评估器结合大模型处理
- 评估指标打分



案例2：代码生成Agent - 评估前后对比

问题示例：“实现用户登录API” 生成代码

评估前

```
def login(username, password):  
  
    query = f"SELECT * FROM  
users  
  
    WHERE  
name='{username}'"  
  
    result = db.execute(query)  
  
    if result and result[0][2] ==  
password:  
  
        return {"status": "success"}
```

SQL注入风险

明文密码对比

无错误处理

无类型注解

无文档

评估后

```
def login(  
    username: str,  
    password: str  
) -> Dict[str, str]:  
  
    """验证用户登录  
Returns: {"status": "success/fail"}  
    """  
  
    try:  
        query = "SELECT * FROM users  
                WHERE name = ?"  
        result = db.execute(query,  
                             (username,))  
  
        if result and  
            check_password_hash(  
                result[0]['password_hash'],  
                password):  
            return {"status": "success"}  
  
    except Exception as e:  
        logger.error(f"Login error: {e}")  
    return {"status": "fail"}
```

参数化查询（防SQL注入）

密码哈希验证

异常处理

类型注解

文档字符串

自动评估发现（秒级）：

LLM-as-Judge: 0.35 → 0.95

案例2：代码生成 - 评估指标改进

12% → 95%

安全问题检出率

SQL注入、硬编码等

0.58 → 0.92

代码质量评分

LLM-as-Judge平均分

30min → 5min

人工Review时间

自动化预检

85% → 98%

pass@5 (能力)

生成正确代码概率



增强提示词

- 添加安全检查清单
- 明确要求参数化查询、异常处理
- 提供安全代码示例



自动化评估

- 每次生成后立即运行6个评估器
- 发现问题自动重新生成（最多3次）
- 记录所有问题用于持续改进



持续监控

- 采样10%生成代码进行深度评估
- 每周生成安全报告
- 发现新模式立即更新评估器

案例3：客服Agent - 用户满意度下降



场景：电商售后客服

- 平台：处理退款、换货、投诉
- 规模：日均5000+对话
- 目标：解决问题+保持用户满意度

业务影响

- 用户满意度 (CSAT) 下降：4.2 → 3.6 (5分制)
- 转人工比例增加：15% → 28%
- 平均对话轮数增加：6 → 9.5 (效率降低)

发现的问题（无评估时）

1. 同理心缺失（28%对话）：

用户："我很生气，等了两周货还没到！"

Agent："请提供订单号以便查询。"

直接要信息，没有表达理解

2. 重复询问（35%对话）：

用户在第2轮已提供订单号

Agent在第5轮又问"请提供订单号"

缺少会话记忆

3. 解决方案不清晰（42%对话）：

用户："什么时候能到账？"

Agent："已为您处理退款。"

缺少关键信息（时间、金额、方式）



案例3：客服Agent - 通过评估找到问题

评估设置（多轮对话评估）

Trace Level (LLM-as-Judge)

- Empathy Score: 共情分数 (0-1)
- Information Completeness: 信息完整性
- Clarity Score: 清晰度分数

Session Level (Custom)

- Context Retention: 是否记住之前说的话
- Efficiency: 对话轮次（越少越好）
- Resolution Quality: 问题解决质量

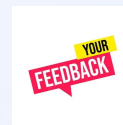


提示词优化

- 添加“从用户消息中提取所有关键信息”指令
- 要求始终包含具体数字（金额、时间）
- 添加共情话术模板

多轮评估

- 不只评估单次响应，评估整个对话
- 检测是否重复询问已提供的信息
- 评估最终是否真正解决问题



实时反馈

- 对话结束后自动运行评估
- 低分对话标记给质量团队审查
- 每周生成改进建议



案例3：客服Agent - 评估前后对话质量

评估前

用户: "我要退款, 订单#12345"

Agent: "好的, 我可以帮您处理退款。
请提供订单号。"

用户已提供订单号, 还要再问

用户: "我刚说了, #12345!"

Agent: "抱歉。订单#12345已处理
退款, 请等待。"

没有说明退款时间和金额

用户: "什么时候到账? 多少钱?"

Agent: "请查看订单详情。"

没有直接回答问题

评估后

用户: "我要退款, 订单#12345"

Agent: "理解您的需求。让我为您
查询订单#12345的退款情况。"

直接使用用户提供的订单号

[调用 verify_order, fetch_policy,
process_refund 工具]

Agent: "已为您处理退款:

- 订单金额: ¥299
- 退款金额: ¥299
- 退款方式: 原路返回支付宝
- 预计到账: 3-5个工作日

退款确认邮件已发送至您的邮箱。

还有其他需要帮助的吗?"

信息完整、清晰

评估结果对比

	评估前	评估后
Empathy (同理心)	0.3	0.95
Clarity (清晰度)	0.4	1.0
Efficiency (效率)	0.5	0.95
用户满意度	2.8	4.6
对话轮数	8	3
升级至人工	Y	N

案例3：客服对话 - 评估指标改进

3.6 → 4.6

用户满意度(CSAT)

5分制, 提升28%

9.5轮 → 4.2轮

平均对话轮数

效率提升56%

28% → 12%

升级至人工率

降低57%

62% → 94%

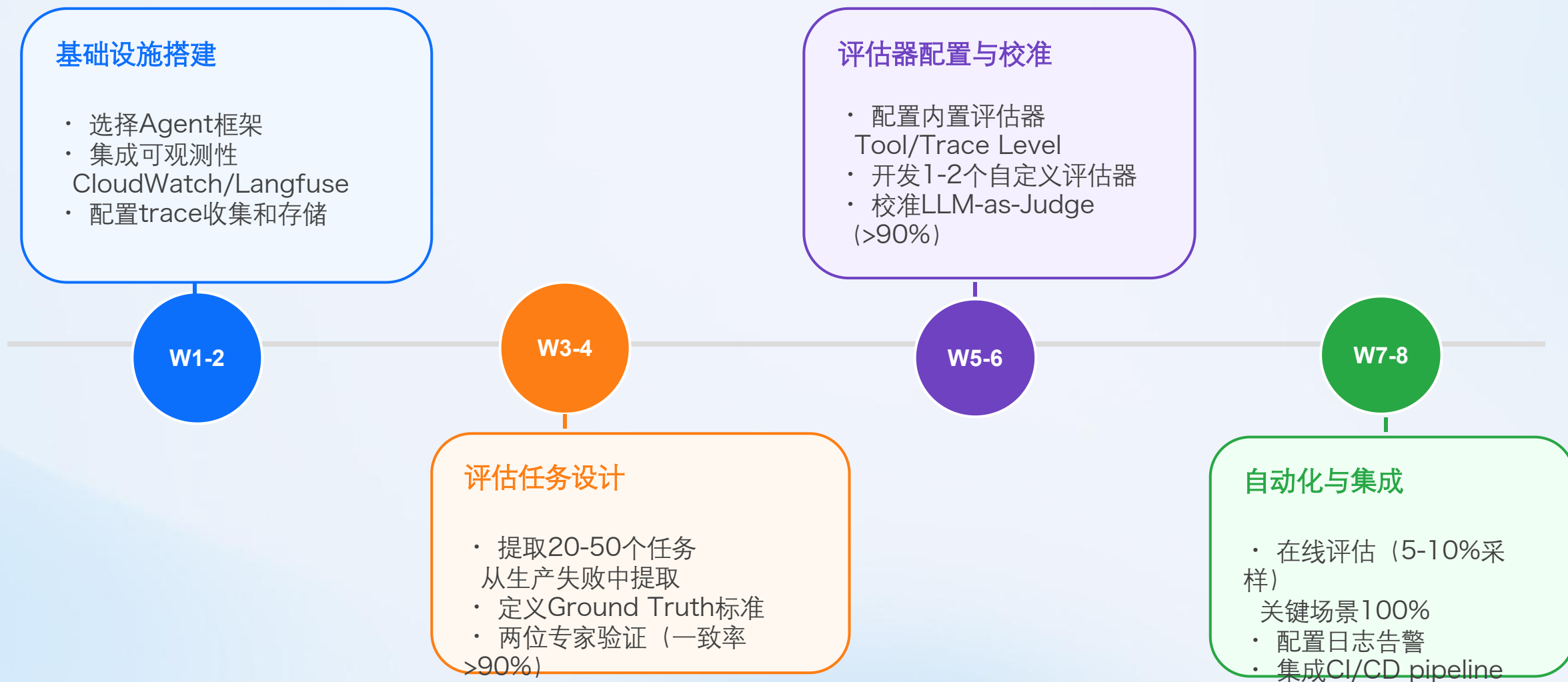
pass¹⁰ (可靠性)

一致性大幅提升

05

工程实践与落地路径

构建评估体系：8周路线图





三个案例的评估方案总结

对比维度	旅游Agent	代码生成	客服对话
核心问题	工具选择失效	安全 + 质量	同理心 + 清晰度
评估框架	AgentCore	Langfuse	Langfuse
Ground Truth	70%	40%	30%
LLM-as-Judge	30%	60%	70%
关键指标	ToolSelection	安全检出率	CSAT满意度
改进效果	67% → 92%	12% → 95%	3.6 → 4.6
检测时间	4周 → 2小时	Review时 → 秒级	用户投诉 → 实时
ROI	>100:1	>50:1	>80:1



评估常见问题

5个常见问题 & 解决方案

问题1 任务设计质量差

→ 两人独立评估，一致率 >90%

问题2 过度依赖LLM-as-Judge，成本过高（\$0.02-0.05/次）

→ 推荐：70% GT + 30% LLM 混合方案

问题3 忽略非确定性

→ $\text{pass}@k$ （能力）vs pass^k （可靠性）多轮评估

问题4 评估与开发脱节

→ CI/CD集成 + 自动告警 + PR门控

问题5 采样策略不当

→ 5-10%常规采样，100%关键场景

评估成本分析

成本分析（三个案例）

旅游Agent

\$300/月 (AgentCore+CloudWatch)
+ \$150/月 (评估执行5%采样) = \$450/月

代码生成

\$100-500/月 (Langfuse+LLM, 10%采样)

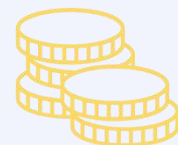
客服对话

\$500-1500/月 (多轮评估, 10%采样)

ROI计算 (旅游Agent示例)

成本: \$450/月
收益: 节省\$20K工程时间 + 避免\$50K+用户流失
+ 15%质量持续提升 (持续价值)

ROI: >100:1 第一个月即回本



假设10万次/天 Agent 调用

- 每天 10万次 Agent 调用
- 每月 300万次调用
- 每次评估 = 1次 LLM 调用 (Judge 模型对 Agent 输出打分)

单次 LLM-as-Judge 的成本:

- 评估 Prompt (评分标准、指令): 约 500-800 tokens
- 被评估的 Agent 输入+输出: 约 500-2000 tokens
- Judge 模型的输出 (评分+解释): 约 200-500 tokens

单次评估大约 1200-3300 tokens (input + output)

Claude Opus/GPT-4: 单次评估 \$0.02-0.05

评估方案	月成本	说明
无评估	0	问题检测需要 4-7 周
100% Ground Truth	0	快但只能评估确定性任务
100% LLM-as-Judge	\$60,000-150,000	太贵
10% 采样 + 混合方案 (30%GT + 70% LLM)	\$1,800-4,500	推荐方案

06

总结

● 核心要点：三个案例的启示

评估是Agent生产化的必要条件

旅游：7周→1天（98.5%时间节省）

代码生成：30min人工review→5min（自动化预检）

客服对话：用户满意度3.6→4.6（28%提升）

方法论：Ground Truth+ LLM-as-Judge

三个层次：Output, Trace, Session

三个阶段：能力评估 → 回归评估 → 生产监控

处理非确定性：pass@k（能力）vs pass^k（可靠性）

不同侧重评估方向

旅游：工具选择（Tool Level重点）

代码生成：质量评估（40% GT + 60% LLM）

客服对话：交互质量（70% LLM + Session Level）

立即行动



- 识别关键场景（20-50个任务）
- 配置基础评估器（内置+自定义）
- 从5-10%采样开始
- 设置CloudWatch告警
- 8周即可建立完整评估体系

评估不是成本，是投资

THANKS

大模型正在重新定义软件

Large Language Model Is Redefining The
Software



AiCon

全球人工智能开发与应用大会

上海站

探索 AI 应用边界

Explore the limits of AI applications

2026 年 6 月 26-27 日 / 上海张江科学会堂



参会咨询

专题：世界模型与多模态智能突破

专题出品人：朱政

极佳视界 / 联合创始人 & 首席科学家



专题：Agent 系统架构与工程化实践

专题出品人：鲁浩楠

OPPO / 大模型算法负责人



专题：AI 原生数据工程

专题出品人：王彦辉

火山引擎 / 数智平台产品总监



专题：Agent 安全、评测与可信治理

专题出品人：马传雷（岳立）

支付宝 / 业务风险技术部负责人



专题：Agent 数据、记忆与运行时基础设施

专题出品人：邓亚峰

EverMind / CEO



专题：企业级研发体系的重构

专题出品人：沈浪

快手 / 研发效能负责人

